

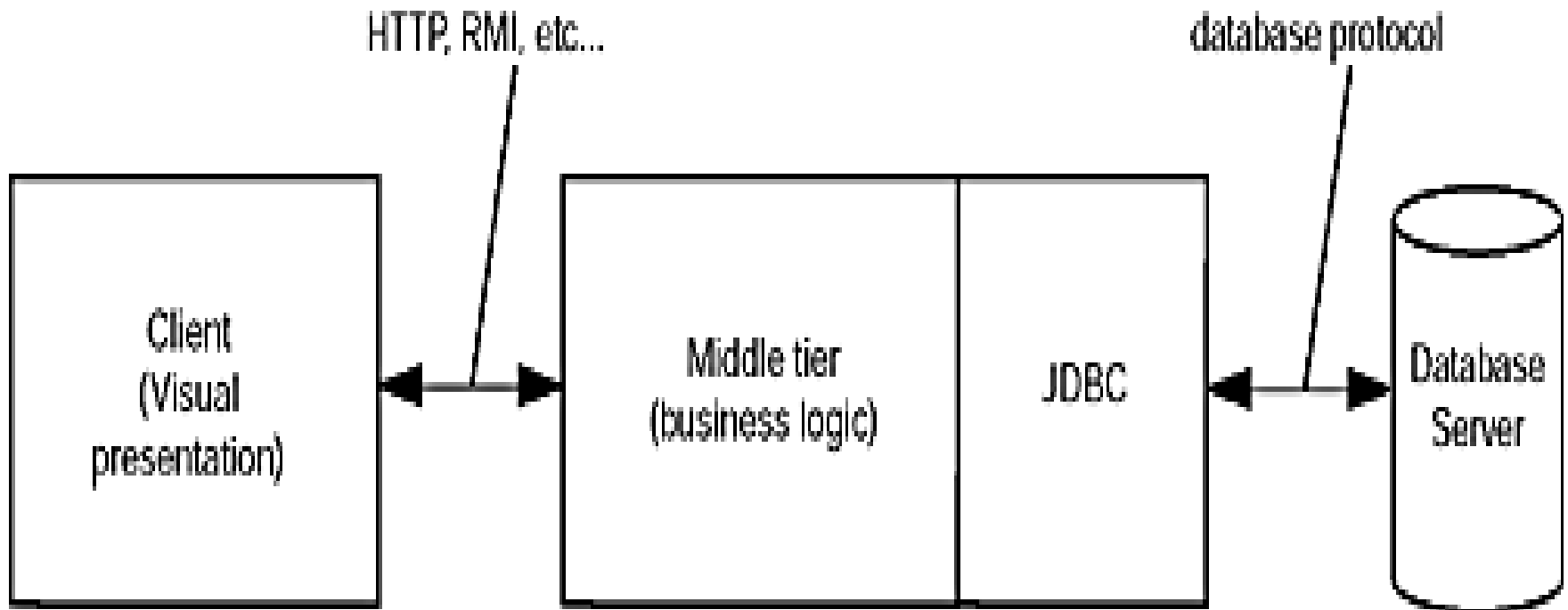
Chapter Four

Introduction to JDBC Programming

Uses of JDBC

- You can use JDBC in both **applications** and **applets**.
- The three-tier model has a couple of advantages.
 - It separates *visual presentation* (on the client) from the *business logic* (in the middle tier) and *the raw data* (in the database).
- Communication between the client and middle tier can occur through
 - **HTTP** (when you use a web browser as the client),
 - **RMI** (when you use an application or applet or another mechanism).
- **JDBC** is used to manage the communication between the middle tier and the back-end database.

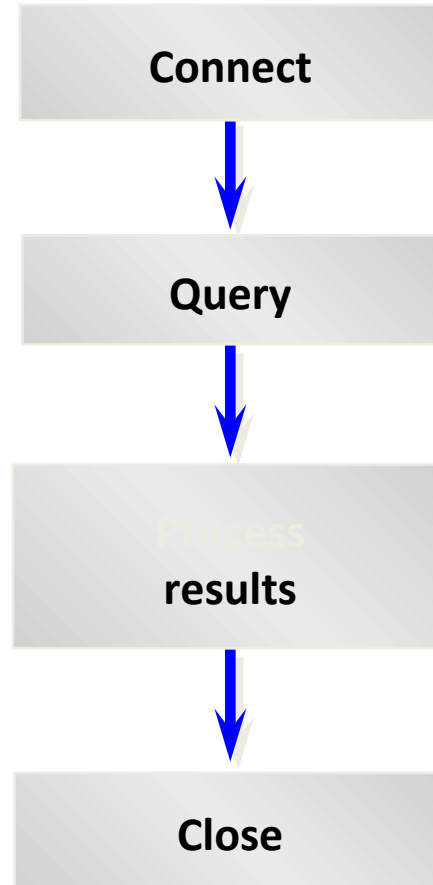
A three-tier application



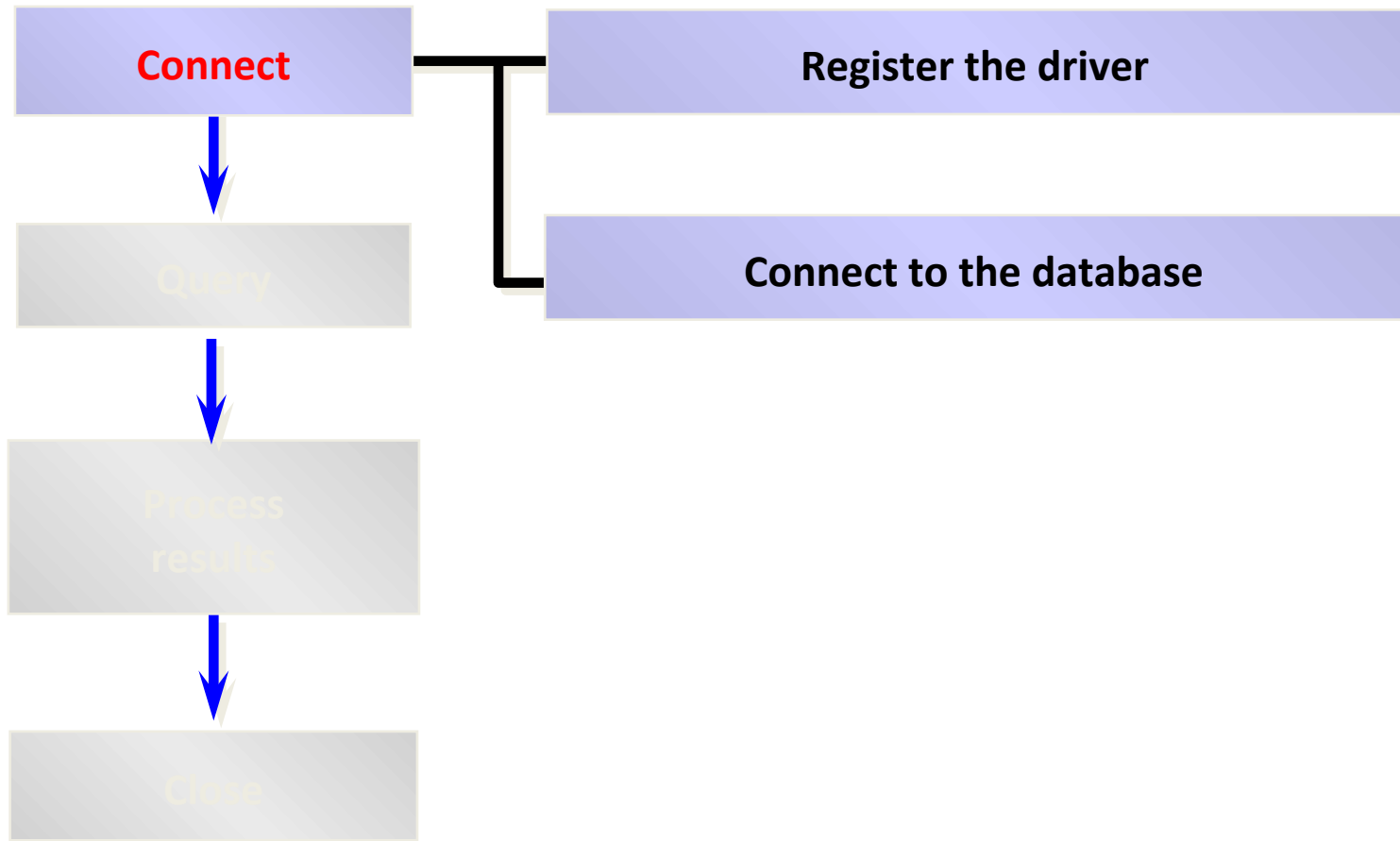
Accessing Database

1. Load the **driver**
2. Creating a **connection** object
3. Create a **statement** object
4. **Execute SQL** query and **get results** using the **ResultSet** object
5. **Close connections**

Overview of Querying a Database With JDBC

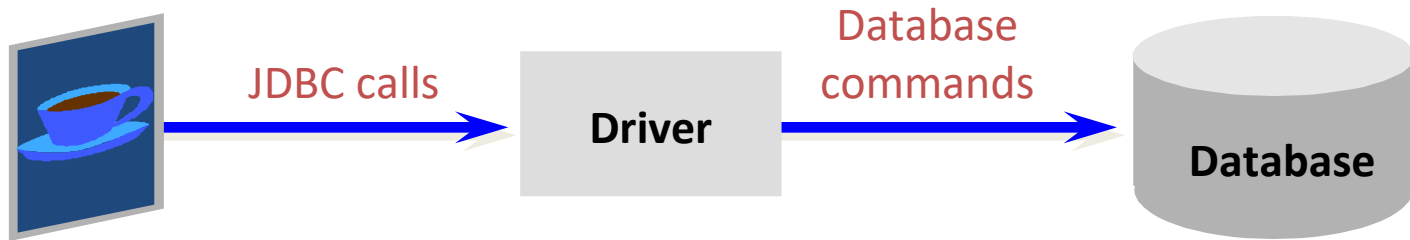


Stage 1: Connect



A JDBC Driver

- Is an interpreter that translates JDBC method calls to vendor-specific database commands



- Implements interfaces in **java.sql**
- Can also provide a vendor's extensions to the JDBC standard
- Registering a Driver**
 - statically load driver

```
Class.forName("foo.bar.MyDriver");
```

```
Connection c = DriverManager.getConnection(...);
```

A JDBC Driver



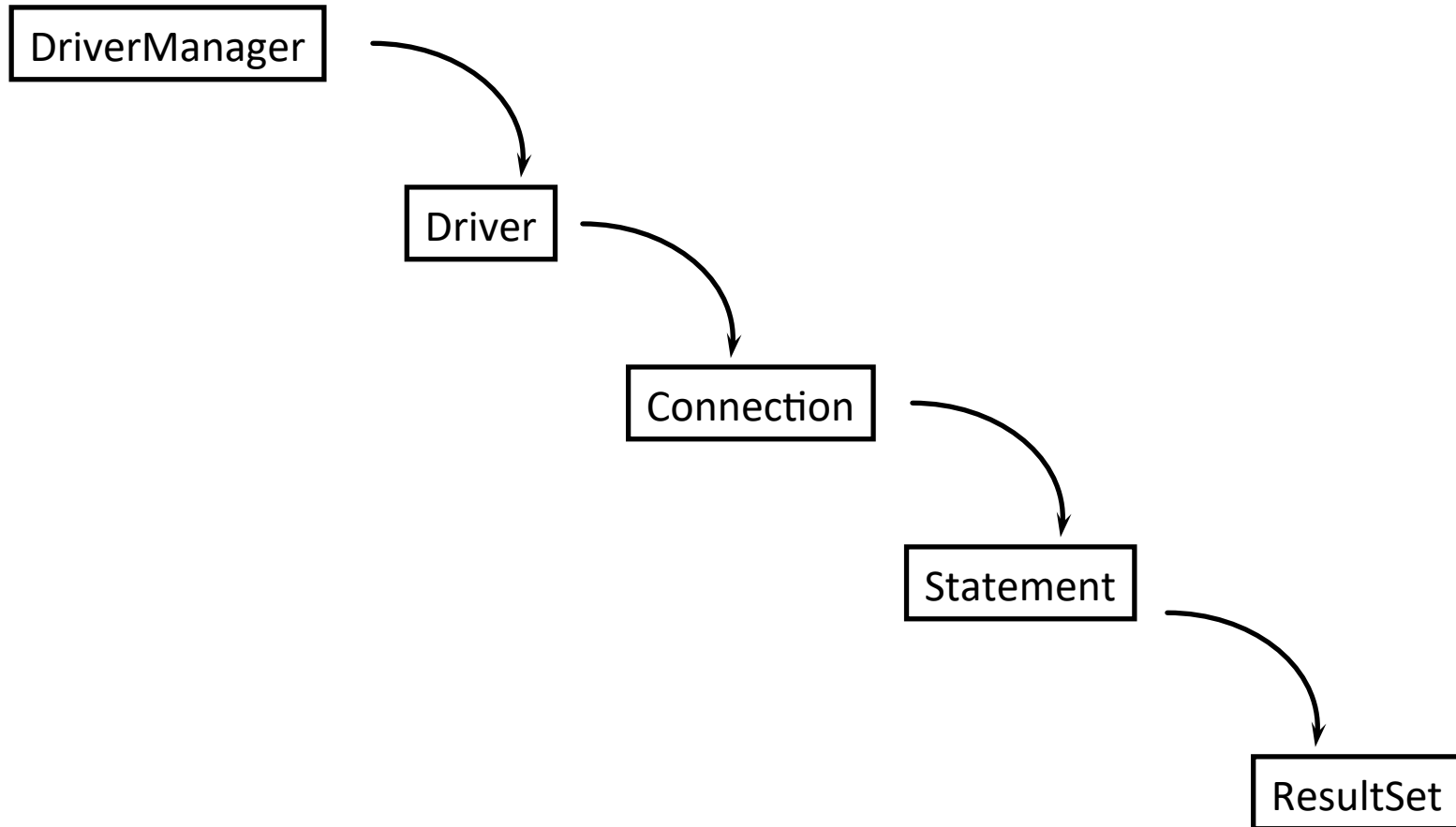
Some JDBC drivers

Driver class	Start of dbURL	Database
sun.jdbc.odbc.JdbcOdbcDriver	jdbc:odbc:	Bridge to Microsoft ODBC (included with JDK)
jdbc.idbDriver	jdbc:idb:	Instant Database
oracle.jdbc.Driver.OracleDriver	jdbc:oracle:thin:@server:port#:dbname	Oracle
postgresql.Driver	jdbc:postgres://host/database	PostgreSQL (freeware database;)
java.sql.Driver	jdbc:mysql://host/database	MySQL (freeware database;)
com.microsoft.jdbc.sqlserver.SQLServerDriver	jdbc:sqlserver:// [serverName[instanceName]][:portNumber] [;property=value[;property=value]]	SQL Server 2012

JDBC Object Classes

- DriverManager
 - Loads, chooses drivers
- Driver
 - connects to actual database
- Connection
 - a series of SQL statements to and from the DB
- Statement
 - a single SQL statement
- ResultSet
 - the records returned from a Statement

JDBC Class Usage



JDBC URLs

- To open a database, you must create a “**database URL**” that specifies:
 - That you’re using JDBC with “**jdbc.**”
 - The “**subprotocol**”: the name of the driver or the name of a database connectivity mechanism.
 - the first subprotocol available is the “**jdbc-odbc bridge,**”
 - The database identifier.
- **jdbc:subprotocol:source**
- each driver has its own subprotocol
- each subprotocol has its own syntax for the source
 - jdbc:odbc:DataSource*
 - e.g. *jdbc:odbc:Northwind*
 - jdbc:mysql://host[:port]/database*
 - e.g. *jdbc:mysql://localhost:3306/accounting*

DriverManager

- When you're ready to connect to the database,
 - call the **static** method **DriverManager.getConnection()**
 - and pass it the following to get into the database.
 - the database **URL**,
 - the **user name**, and
 - a **password**

Connection getConnection (String url, String user, String password)

- Connects to given JDBC URL with given user name and password
- Throws **java.sql.SQLException**
- returns a **Connection object**

Connection

- A Connection represents a session with a specific database.
- Within the context of a Connection, SQL statements are executed and results are returned.
- Can have multiple connections to a database
- **Obtaining a Connection**

```
String url = "jdbc:odbc:Northwind";  
try {  
    Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");  
    Connection con = DriverManager.getConnection(url);  
}  
catch (ClassNotFoundException e) {  
    e.printStackTrace();  
}  
catch (SQLException e){  
    e.printStackTrace();  
}
```

Statement

- Once the connection is made, you can use the resulting **Connection** object to create a **Statement** object using the **createStatement()** method.
- A **Statement** object is used for executing a static SQL statement and obtaining the results produced by it.
- **Statement Methods**
 - **ResultSet executeQuery(String)** - Execute a SQL statement that returns a *single ResultSet*.
 - **int executeUpdate(String)** - Execute a SQL INSERT, UPDATE or DELETE statement. Returns the number of rows changed.
 - **boolean execute(String)** - Execute a SQL statement that may return *multiple results*.

ResultSet

- A **ResultSet** provides access to a table of data generated by executing a Statement.
- Only one **ResultSet** per Statement can be open at once.
- The table rows are retrieved in sequence.
- A **ResultSet** maintains a cursor pointing to its current row of data.
- The 'next' method moves the cursor to the next row.
 - you can't rewind

ResultSet Methods

- **boolean next()**
 - activates the next row
 - the first call to next() activates the first row
 - returns false if there are no more rows
- **void close()**
 - disposes of the ResultSet
 - allows you to re-use the Statement that created it
 - automatically called by most Statement methods

ResultSet Methods

- *Type* *getType*(int columnIndex)
 - returns the given field as the given type
 - fields indexed starting at 1 (not 0)
- *Type* *getType*(String columnName)
 - same, but uses name of field
 - less efficient
- int *findColumn*(String columnName)
 - looks up column index given column name

ResultSet Methods

- String getString(String columnName)
- boolean getBoolean(String columnName)
- byte getByte(String columnName)
- short getShort(String columnName)
- int getInt(String columnName)
- long getLong(String columnName)
- float getFloat(String columnName)
- double getDouble(String columnName)
- Date getDate(String columnName)
- Time getTime(String columnName)
- Timestamp getTimestamp(String columnName)

isNull

- In SQL, **NULL** means the field is empty, not the same as **0** or “ ”
- In JDBC, you must explicitly ask if a field is null by calling **ResultSet.isNull(column)**

- **Sample Database**

Employee ID	Last Name	First Name
1	Davolio	Nancy
2	Fuller	Andrew
3	Leverling	Janet
4	Peacock	Margaret
5	Buchanan	Steven

SELECT Example

```
Connection con = DriverManager.getConnection(url, "alex", "8675");
Statement st = con.createStatement();
ResultSet results = st.executeQuery("SELECT EmployeeID, LastName,
                                   FirstName FROM Employees");
while (results.next()) {
    int id = results.getInt(1);
    String last = results.getString(2);
    String first = results.getString(3);
    System.out.println("'" + id + ": " + first + " " + last);
}
st.close();
con.close();
```

Mapping Java Types to SQL Types

SQL type

CHAR, VARCHAR, LONGVARCHAR

NUMERIC, DECIMAL

BIT

TINYINT

SMALLINT

INTEGER

BIGINT

REAL

FLOAT, DOUBLE

BINARY, VARBINARY, LONGVARBINARY

DATE

TIME

TIMESTAMP

Java Type

String

java.math.BigDecimal

boolean

byte

short

int

long

float

double

byte[]

java.sql.Date

java.sql.Time

java.sql.Timestamp

Example – Connection (MySQL)

```
import java.sql.*;
```

Importing java.sql.* that contains all the classes we need

```
public class SQLConnect {  
    Connection conn = null;  
    Statement stmt = null;  
    ResultSet rs = null;
```

Connection, Statement and ResultSet are defined as class variables

```
public SQLConnect() {}  
public void createConnection() {  
    try{
```

```
        Class.forName("com.mysql.jdbc.Driver").newInstance();
```

```
    }  
    catch (Exception E) {  
        System.out.println(E);  
    }
```

Dynamically loading the specific JDBC driver. The runtime environment must know where the library is located!

```
    try{  
        conn =
```

```
        DriverManager.getConnection("jdbc:mysql://localhost/mytest,user=test  
master,password=1234");
```

Connecting to the database using the url

```
    }  
    catch (SQLException E) {  
        System.out.println(E);  
    }
```

```
}
```


Example – Access and Query

```
public String getPhones(){
```

```
String output = "";
```

```
try {
```

```
    stmt = conn.createStatement();
```

```
    rs = stmt.executeQuery("SELECT * FROM phones");
```

```
    if (rs != null){
```

```
        while (rs.next()){
```

```
            output += rs.getString("phone") + "\n";
```

```
        }
```

```
    }
```

```
}
```

```
catch (Exception E){
```

```
    System.out.println(E.getMessage());
```

```
}
```

Creating a statement

Creating a ResultSet, based on a SQL statement

Going through the ResultSet by using rs.next(). Remember – you need to call the next method before you start reading from the ResultSet

Reading a field from the ResultSet

Example – Cleaning off

```
finally {  
    if (rs != null) {  
        try {  
            rs.close();  
        }  
        catch (SQLException sqlEx) {}  
        rs = null;  
    }  
    if (stmt != null) {  
        try {  
            stmt.close();  
        }  
        catch (SQLException sqlEx) {}  
        stmt = null;  
    }  
    return output;  
}
```

Cleaning off is best done in the “finally” clause

Cleaning off ResultSet

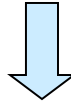
Cleaning off Statement, after the ResultSet

Cleaning off the connection, in a different method (why?)

```
public void closeConnection(){  
    if (conn != null){  
        try {  
            conn.close();  
        }  
        catch (SQLException sqlEx) {}  
        conn = null;  
    }  
}
```

Example – Test Client

```
public class Test {  
    public static void main(String[] args) {  
        SQLConnect connect = new SQLConnect();  
        connect.createConnection();  
        String allPhones = connect.getPhones();  
        connect.closeConnection();  
        System.out.println("phones:");  
        System.out.println(allPhones);  
    }  
}
```



Output
phones:
+972-4-9831894

Modifying the Database Contents

- To change the contents of the database we shall have to submit our SQL statements via the *executeUpdate* method.
- Examples

(i) String `insert` = "INSERT INTO Accounts"+ " VALUES (123456, 'Smith','"+ " 'John James',752.85)";

int result = statement.executeUpdate(insert);

(ii) String `change` = "UPDATE Accounts"+ " SET surname = 'Bloggs','"+ " firstName = 'Fred Joseph'" + " WHERE acctNum = 123456";

statement.executeUpdate(change);

(iii) String `remove` = "DELETE FROM Accounts"+ " WHERE Balance < 100";

result = statement.executeUpdate(remove);

- The integer returned is often used to check whether the update has been carried out.
- Example

```
int result = statement.executeUpdate(insert);  
if (result==0)  
    System.out.println("* Insertion failed! *");
```

The PreparedStatement Class

- With **PreparedStatement** you can **create SQL with parameters that are dynamically passed by the program.**
- Unlike **Statement** class **PreparedStatement** compiles the **sqlQuery** only once.
- The parameters are provided by the appropriate **setXXX()** method depending on the data type.
- The SQL statement may have several parameters (question marks), and the first argument of the setter enables you to specify each parameter's number.

- **For example:**

```
PreparedStatement stmt=conn.prepareStatement(
    "SELECT * from Employee WHERE empno=?
    and
    ename=?");
for (int i=0;i<empNumbers.length; i++){
    stmt.setInt(1,empNumbers[i];)
    stmt.setString(2,empNames[i];)
    stmt.executeQuery(sqlQuery);
}
```

Scrollable Result Sets and RowSet

- create a scrollable result set so the cursor can be moved back and forth if need be
- There is a two-argument version of the method **createStatement()**.
- The first argument specifies the type of scrolling
 - (TYPE_FORWARD_ONLY,
 - TYPE_SCROLL_INSENSITIVE, or
 - TYPE_SCROLL_SENSITIVE)
- The second makes the result set updateable or read-only
 - CONCUR_READ_ONLY or
 - CONCUR_UPDATABLE

- For example,
Statement stmt = con.createStatement
(ResultSet.TYPE_SCROLL_INSENSITIVE,
ResultSet.CONCUR_READ_ONLY);
ResultSet rs = stmt.executeQuery(“SELECT * from Employee”);
- The TYPE_FORWARD_ONLY parameter allows only forward movement of the cursor.
- The difference between TYPE_SCROLL_INSENSITIVE and TYPE_SCROLL_SENSITIVE is in whether scrolling reflects changes that have been made to the result set.

- Example to set the cursor at the end of the result set and moves the cursor backward

```
rs.afterLast();  
while (rs.previous()){  
    int empNo = rs.getInt("EMPNO");  
    String eName = rs.getString("ENAME");  
    String job = rs.getString("JOB");  
    System.out.println(" "+ empNo + ", " + eName + ", " + job_title);  
}
```

- You can also move the cursor to a specific row by using the following self-explanatory methods:
 - **rs.absolute(25);** // moves the cursor to the 25th row
 - **rs.relative(-4);** // moves the cursor to the 21st row
 - **rs.first();**
 - **rs.last();**
 - **rs.beforeFirst();**

- If the result set is updatable (CONCUR_UPDATABLE) you can modify the underlying database table while scrolling.
- For example, the following statements will update the job title of the employee based on the current cursor's position:
 - `rs.updateString("JOB_TITLE","Manager");`
 - `rs.updateRow();`
- Scrollable result sets enable you to traverse the result set in both directions, but they have a drawback:
 - They hold the database connection, which may be required by another thread or program.
- The package **javax.sql** includes the interface **RowSet**, which is a subclass of **ResultSet**.
- **RowSet** gets the data from the database, then disconnects, but still allows Java to work with the data.